

Lab Guide

Neural-Network Demand Forecasting in Google Colab

Every cell of the notebook explained in full sentences, with the questions we raise in class and their answers, for working on your own or in pairs.

Dr. Hyunwoong Ko

School of Manufacturing Systems and Networks
Office ISTB12 355J

How to Use This Guide

This guide accompanies the hands-on lab of Lecture 1, Part 6. In the lab you build, train, and judge the neural network that we have been drawing in class for two weeks: twelve months of demand plus twelve month-of-year switches go in, and a forecast for the next month comes out. The building happens in a Google Colab notebook, cell by cell. This guide explains every one of those cells in full sentences, so that you can work through the notebook on your own or with a partner without needing someone to narrate it for you.

You do not need any programming background to follow this guide. Every piece of Python that appears in the notebook is explained the first time you meet it, and Part C collects all of that vocabulary in one place so you can look things up. The course policy applies here as it does everywhere in MFG 510: the Python code is the case material of this module, and the exams assess models, algorithms, and pseudocode, never software syntax. What you must understand are the ideas behind each step, and those are exactly what this guide spells out.

Three kinds of boxes appear throughout the guide:

- A **code box** (gray, with a maroon edge) shows the code of a notebook cell exactly as it appears in Colab. Blanks that you must fill are shown as four underscores, `_____`, highlighted in gold.
- A **note box** (gray, with a maroon edge, labeled) gives a checkpoint, a hint for a blank, a lecture link, or a common mistake.
- A **question box** (gold) asks one of the questions we raise in class for you to think about, and then gives the answer in full. Read the question, try to answer it in your own words first, and then compare with the answer. These are the thinking questions of the lab.

One thing is deliberately not in this guide: the answers to the blanks in the notebook. The CHECK cell that follows every blank tells you whether your answer is right, and we go over all of the answers together in the debrief at the end of the lab.

What You Need

- A laptop with a web browser and an internet connection. Nothing is installed on your computer; everything runs in the cloud.

- A Google account. Your ASU Google account normally works with Colab; if it is blocked, use any personal Gmail account. Colab is free.
- Two files from Canvas: the notebook `MFG510_NN_Forecasting_Colab.ipynb` and the data file `demand_monthly.csv`. Download both to your computer before you begin.
- Optionally, the lab slides (`MFG510_Lab_NN_Forecasting_Colab.pdf`), which this guide follows stage by stage.

Contents

Part	Title	What it covers
Part A	Before you start	What Colab and a notebook are, how to get set up, how to read an error message, and how the lab session runs.
Part B	The problem and the data	What we are forecasting, what the data file contains, and how this connects to TES and regression from the lecture.
Part C	Just enough Python	Every programming word used in the notebook, explained in one table each for Python, pandas/numpy, and Keras.
Part D	The notebook, cell by cell	The core of the guide: all twenty cells explained line by line, with checkpoints, hints for the blanks, lecture links, and the thinking questions with answers.
Part E	Reading your results like a forecaster	How to state a scorecard in the language of Lecture 1, Part 2, and why your numbers differ from your neighbor's.
Part F	Troubleshooting	A table of the error messages you may see, what they mean, and what to do.
Part G	Vocabulary and what is examinable	The ten code words of the day mapped to lecture concepts, a glossary, and the list of examinable ideas.

The Lab in One Page

The whole lab is six stages, and each stage is a handful of notebook cells. Six of the cells contain blanks marked **YOUR TURN**; every one of them is followed by a **CHECK** cell that prints a check mark when your answer is right.

Stage	Name	Cells	What happens
1	Load and look	1 to 4	Import the libraries, read the data file, add a month index, and plot the series. Always look at the data before modeling it.
2	Build the training table	5 (YOUR TURN)	Turn one time series into a table of examples: twelve lag columns, twelve month switches, and one label per row. This is the sliding window from the lecture diagram.
3	Split by time	6 (YOUR TURN) and 7	Keep the most recent 20 percent of months as the test set. Never split time-series data at random.

Stage	Name	Cells	What happens
4	Normalize and build	8, 9, 10 (YOUR TURN)	Rescale the inputs, then build the network from the lecture: 24 inputs, two hidden layers of 64 units, one output.
5	Train	11 and 12	Run gradient descent with early stopping and read the learning curve.
6	Score, discuss, challenge	13 (YOUR TURN) to 18 (YOUR TURN)	Judge the network with MAD and MAPE against two free baselines, discuss why a free baseline is so strong, and then retrain the network to correct the baseline. Cells 19 and 20 are an optional experiment helper for those ahead of pace.

How the Lab Session Runs

The class is a guided lab, not a lecture. For the first twelve minutes or so, the instructor frames the problem, the data, the setup, and the notebook map (Slides 1 to 8), and then stops talking. From then on you build, working in pairs, cell by cell. The instructor circulates, so raise a hand whenever you are stuck. At about minute 55 the class comes back together for the debrief: the answers to the blanks, the scorecard, the three discussion questions, and a live run of the challenge. The last five minutes are the wrap-up: save your notebook, because a follow-up assignment will build on it (details will be posted on Canvas).

Pace check: By minute 25 you should have reached Cell 10. By minute 40 you should have reached Cell 13. If you are ahead of that pace, go on to the challenge in Cell 18.

Questions after class: Post them on Piazza. The Piazza link will be shared right after class. Questions are answered within 24 hours, and answering a classmate's question earns the Piazza bonus. Office hours continue as usual.

Part A — Before You Start

A1. What Google Colab is, and what a notebook is

Google Colab (short for Colaboratory) is a free website, colab.research.google.com, that lets you run Python code in your browser. The code does not run on your laptop; it runs on a computer in Google's cloud, which Colab calls the *runtime*. That is why nothing needs to be installed: TensorFlow, pandas, numpy, and matplotlib, the four libraries this lab uses, are already there.

The document you open in Colab is a *notebook*, a file ending in `.ipynb`. A notebook is a sequence of *cells*, and there are two kinds:

- A **text cell** contains explanations written for you to read. You do not run text cells. In our notebook, every code cell is preceded by a text cell that names it (Cell 1, Cell 2, and so on) and says what it does.
- A **code cell** contains Python code. You run it, and its output (printed text, a table, or a plot) appears directly underneath it.

To run a code cell, click inside it and either press the round play button that appears at its left edge, or press Shift and Enter together. While the cell is running, the play button turns into a spinning icon; when it finishes, a number in square brackets such as `[3]` appears at the left, telling you that this was the third cell you ran. Cells are meant to be run from top to bottom, one after another, because each cell builds on what the cells above it created. If you run a cell that refers to something a previous cell was supposed to create, and that previous cell has not been run yet, you get an error that says a name is not defined; the fix is simply to run the earlier cells first.

Everything a cell creates (a table, a trained model) stays in the runtime's memory until the runtime is restarted or disconnects. Colab disconnects a runtime after a long period of inactivity or after about twelve hours; if that happens, the menu Runtime › Run all re-runs every cell from the top, and you will have to upload the data file again, because uploaded files do not survive a restart.

A2. Getting set up (five minutes)

1. Open colab.research.google.com in your browser and sign in with a Google account. If your ASU account is blocked, use a personal Gmail account.
2. Open the notebook: choose File › Upload notebook, and pick the file `MFG510_NN_Forecasting_Colab.ipynb` that you downloaded from Canvas. The notebook opens in a new tab.
3. Run Cell 3. A button labeled Choose Files appears under the cell. Click it and pick `demand_monthly.csv`. The file name must match exactly, because the code looks for a file with that name.
4. Run the cells in order with the play button (or Shift+Enter). Read the text cell above each code cell before running it.
5. When you reach a cell marked **YOUR TURN**, fill every blank (written as `_____`) before running the cell, then run the **CHECK** cell that follows it. A check mark means you can move on. Because of the blanks, Runtime › Run all will stop at the first empty blank; that is expected, not a problem.

Your work is saved automatically to your Google Drive if you choose File › Save a copy in Drive; do that early, so that you do not lose the notebook if the browser tab closes. To download the notebook to your own computer, use File › Download › Download `.ipynb`.

A3. Blanks and CHECK cells

Six cells contain blanks. Every blank asks for a single word, number, or short expression that stands for a *concept* from the lecture, never for a piece of syntax you would have to know by heart. For example, one blank asks how many lag months the window should hold, and another asks which activation function the hand-worked example used. The text cell above each blank cell states exactly what each blank asks, and this guide repeats those hints.

After every blank cell comes a CHECK cell. It contains lines that begin with **assert**; an assert line tests a statement and, if the statement is false, stops with an error message. The messages are written to tell you which blank is wrong and why, for example "expected 12 lag columns — check range(1, N)". If a CHECK cell fails, read its message, fix the blank in the cell above, run that cell again, and then run the CHECK cell again. When all of its assertions hold, the CHECK cell prints a line beginning with a check mark.

A4. How to read an error message

When a cell fails, Colab prints red text under it. The text is a *traceback*: it shows the line of code that failed and, on its last line, the type of error and a short message. The last line is almost always the only one you need. The most common ones in this lab are these:

Last line says	What it means	What to do
<code>NameError: name '____' is not defined</code> or <code>KeyError: '____'</code>	A blank is still empty: the code tried to use four underscores as if they were a name.	Fill the blank and run the cell again.
<code>NameError: name 'dataset' is not defined</code> (or any other name from an earlier cell)	An earlier cell has not been run in this runtime, so the thing it creates does not exist yet.	Run the cells above it from the top, then this one.
<code>FileNotFoundError: demand_monthly.csv</code>	The data file has not been uploaded, or it was uploaded under a different name.	Run Cell 3 again and upload the file; check the exact name.
<code>AssertionError: expected ...</code> in a CHECK cell	Your answer to a blank is wrong; the message says which one.	Read the message, fix the blank in the cell above, run both cells again.
<code>SyntaxError: ...</code>	Something in the cell is not valid Python, usually a missing quotation mark or parenthesis introduced while filling a blank.	Look at the line the arrow points to; make sure text answers are inside quotation marks, exactly as the surrounding code shows.
A message about the runtime disconnecting	The cloud computer went away (inactivity, or a long session).	Runtime › Run all, then upload the CSV again when Cell 3 asks.

Part F at the end of this guide has a longer troubleshooting table.

A5. What to expect from your numbers

Training a neural network starts from random initial weights and shuffles the rows during training, so two people running the same notebook get slightly different results. Your network's MAD will not match your neighbor's to the decimal, and it will not match the numbers printed in this guide exactly. That is normal. What should agree is the *ranking*: which forecast is best, which is worst, and roughly by how much. Whenever this guide gives a number for a trained network, read it as "about this much".

Part B — The Problem and the Data

B1. The problem in words

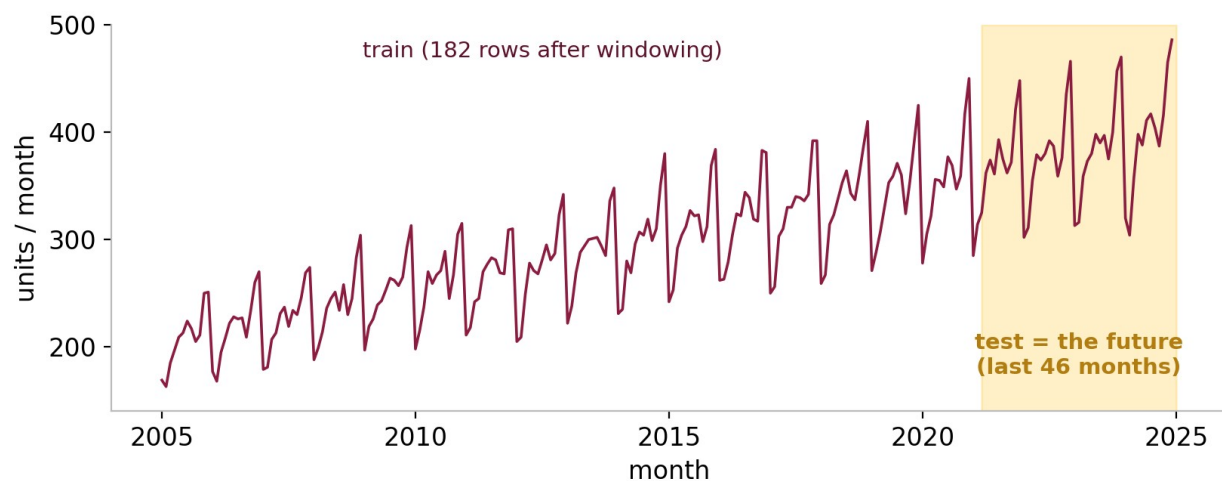
One product family has had its monthly demand recorded since January 2005: two hundred and forty months of history, up to December 2024. Every month, the planning team needs a number for next month's demand, because that number feeds the production plan and the inventory decisions that occupy the rest of this course.

In the notation of the lecture, the task is this: given the demand of the last twelve months, $A(t)$, $A(t-1)$, ..., $A(t-11)$, and the calendar month, produce a forecast for the coming month,

$$\hat{A}(t+1) = f(A(t), A(t-1), \dots, A(t-11), \text{month}; W),$$

where f is the neural network and W is its set of weights. In the lecture we traced such an f by hand for a tiny network; today the computer finds the weights W from the data.

The judging rule is the one from Lecture 1, Part 2 (measuring forecast accuracy): hold out the most recent months, forecast them without looking at them, and compare the forecasts to what actually happened using MAD (mean absolute deviation) and MAPE (mean absolute percentage error). We compare the network against two *free* baselines that need no model at all: the **naïve forecast**, which copies last month's demand, and the **seasonal naïve forecast**, which copies the demand of the same month last year.



The whole series. The gold band is the last 46 months, the future that the model never sees during training and on which it is scored.

B2. The data file: demand_monthly.csv

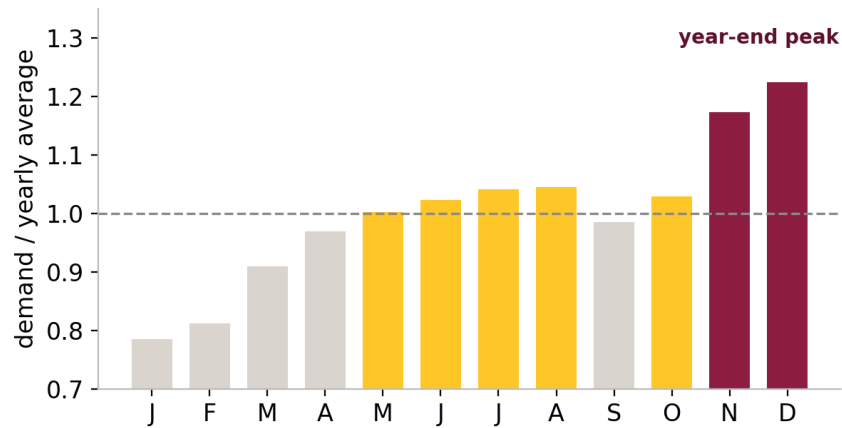
The file is as simple as data gets. It has two columns and two hundred and forty rows:

- **month**, written as text such as 2005-01, 2005-02, ..., 2024-12;
- **demand**, the number of units demanded in that month, a whole number.

A few facts about the series that are worth keeping in your head while you work:

- Demand starts at 169 units in January 2005 and ends at 486 units in December 2024. The lowest value in the file is 163 and the highest is 486.
- The level rises steadily, by about 0.8 units per month, which is roughly 10 units per year, or about 3 to 4 percent per year. That is the **trend**.

- Every year has the same shape: November and December run roughly 20 percent above the year's average, while January and February run roughly 20 percent below it. That is the **seasonality**, and the figure below shows the profile month by month.
- On top of trend and season there are irregular wiggles of about 8 units. That is the **noise**.
- There are no missing values and no outliers. Today is about the model, not about data cleaning.



The seasonal profile: each month's demand divided by the average of its year, averaged over the twenty years. The year-end peak is the dominant feature.

The series is *synthetic*: it was generated from a known recipe (a straight-line trend, a fixed seasonal profile, and random noise). We say this openly because it matters for interpreting the results. It means the answers are clean, the three ingredients are really there to be found, and the two baselines are honest competitors. Real demand data is messier, and one of the discussion questions at the end of the lab asks what the synthetic recipe implies for the strength of the seasonal naive forecast.

Careful. The network never sees this two-column file directly. In Stage 2 we manufacture a new table from it, with one row per month, twenty-four input numbers per row, and one label per row. That manufactured table is what the network learns from. Keep the two tables apart in your mind: the raw series, and the training table built from it.

B3. How this connects to what you already know

To TES (triple exponential smoothing). TES would model this series with a level, a trend, and twelve seasonal factors, updated by three smoothing constants. The network is asked to learn the same three ingredients, but it does so as 5,825 weights fitted from examples, with no formula for level, trend, or season written down anywhere. Whether that flexibility helps or hurts on a series this clean is one of the lessons of the lab.

To regression (Lecture 1, Part 5, associative forecasting). In the Nodel example, a table of inputs (for example, payroll) and one output (sales) was fitted with a straight line, $y = a + b \cdot x$, by least squares. The training table of Stage 2 is exactly such a table, with twenty-four input columns instead of one and 228 rows instead of five or six. The network is a regression with a much more flexible f , and it is fitted by gradient descent instead of by a closed-form formula. The one-hot month switches enter the table the way a categorical predictor enters a regression: as 0/1 dummy variables.

To Lecture 0, Section 5 (gradient descent). Training the network is gradient descent on the loss surface: start from random weights, compute the gradient of the loss with respect to the weights (that is what

backpropagation does), and step downhill with a step size set by the learning rate η . In Cell 11 you watch this happen, and in Cell 12 you see the loss walking downhill epoch by epoch.

To the Forecasting as Learning diagram. The lecture's diagram shows a window of twelve months sliding along the series, each position producing one example (twelve lags and the month as inputs, the next month as the answer). Cell 5 executes that diagram in three lines of code.

Part C — Just Enough Python

You do not need to learn Python to do this lab, but you will read Python, and reading is much easier once a few recurring words are familiar. The three tables below cover every construction that appears in the notebook. Skim them now, and come back to them whenever a line of code looks strange. Part D also explains each line where it first appears.

C1. Python basics

Code	What it means	Where you meet it
<code># some words</code>	A comment. Everything after <code>#</code> on a line is a note for humans; Python ignores it.	Every cell
<code>import numpy as np</code>	Load a library (a collection of ready-made tools) and give it a short nickname. Afterwards <code>np.mean</code> means "the mean function from numpy".	Cell 2
<code>from tensorflow import keras</code>	Load one part of a library, so that you can write <code>keras</code> instead of <code>tensorflow.keras</code> .	Cell 2
<code>x = 5</code>	Assignment: give the name <code>x</code> to the value 5. From then on <code>x</code> stands for that value. A single <code>=</code> always means "store this under that name", not "is equal to".	Everywhere
<code>print(...)</code>	Show something under the cell. Several things separated by commas are printed on one line with spaces between them.	Everywhere
<code>'demand'</code>	Text (a <i>string</i>). Text must be inside quotation marks; single and double quotes work the same. Column names are strings.	Cells 4, 5, 13
<code>f'lag_{k}'</code>	An <i>f-string</i> : text with a value pasted in where the braces are. If <code>k</code> is 3, this is the text <code>lag_3</code> .	Cells 5, 7, 13
<code>[a, b, c]</code>	A <i>list</i> : several values in order, inside square brackets.	Cells 7, 10, 11
<code>for k in range(1, 13):</code>	A loop. <code>range(1, 13)</code> produces the numbers 1, 2, ..., 12 (it stops <i>before</i> the second number), and the indented lines below the colon are run once for each value of <code>k</code> .	Cell 5
<code>def mad(a, f): return ...</code>	Define a function named <code>mad</code> with two inputs, <code>a</code> and <code>f</code> ; <code>return</code> gives back its result. Later, <code>mad(actual, pred)</code> runs it with those two inputs.	Cell 13, Cell 19
<code>len(x)</code>	The number of items in <code>x</code> ; for a table, the number of rows.	Cell 4, Cell 6
<code>int(182.4)</code>	Convert to a whole number by dropping the decimals: 182.	Cell 6
<code>x % 12</code>	The remainder after dividing by 12 (the <i>modulo</i> operation). For <code>t = 0, 1, ..., 11</code> it gives 0, 1, ..., 11; for <code>t = 12</code> it gives 0 again.	Cell 4

Code	What it means	Where you meet it
<code>[f'lag_{k}' for k in range(1, 13)]</code>	A <i>list comprehension</i> : build a list by running the expression once for each <code>k</code> . Here it makes the twelve names <code>lag_1</code> , ..., <code>lag_12</code> . The <code>+</code> between two lists joins them into one longer list.	Cell 7
<code>if not os.path.exists(...):</code>	Run the indented lines only if the condition holds; here, only if the file does not exist yet.	Cell 3
<code>assert condition, 'message'</code>	Check that the condition holds; if it does not, stop with the message. This is how the CHECK cells work.	Every CHECK cell
<code>try: ... except Exception as e:</code>	Try the indented lines; if any of them fails, run the lines under except instead of stopping. Used in the optional export cell so that a missing library does no harm.	Cell 16
<code>a.method(...)</code>	Many values carry their own functions, called <i>methods</i> , reached with a dot: <code>dataset.dropna()</code> , <code>model.fit(...)</code> . Read the dot as "belonging to".	Everywhere

C2. pandas, numpy, and matplotlib (tables, arrays, plots)

Code	What it means	Where you meet it
<code>pd.read_csv('file.csv')</code>	Read a comma-separated file into a DataFrame , which is a table in memory: named columns, numbered rows (the row numbers start at 0). Think of it as a spreadsheet you talk to with code.	Cell 3
<code>raw.tail()</code> and <code>raw.head(2)</code>	Show the last five rows, or the first two rows, of a table. A bare expression on the <i>last</i> line of a cell is displayed automatically; that is why these lines have no print .	Cells 3, 5, 15
<code>dataset['demand']</code>	Select one column by its name. The result is a Series , a single column with its row numbers.	Cells 4, 5
<code>dataset['t'] = ...</code>	Create a new column named t (or overwrite it) by assigning to it.	Cells 4, 5
<code>dataset[feature_cols]</code>	Select several columns at once by giving a list of names. The result is a smaller table.	Cell 7
<code>raw.copy()</code>	Make an independent copy of a table, so that changing the copy leaves the original untouched.	Cell 4
<code>dataset.isna().sum()</code>	Count the missing values in each column (isna marks missing cells, sum counts them).	Cell 4
<code>np.arange(240)</code>	The numbers 0, 1, 2, ..., 239 as an array.	Cell 4
<code>dataset['demand'].shift(k)</code>	The same column moved <i>down</i> by <code>k</code> rows, so that each row now holds the value from <code>k</code> rows earlier. The first <code>k</code> rows have nothing to hold and become NaN (Not a Number, meaning empty).	Cell 5

Code	What it means	Where you meet it
<code>pd.get_dummies(..., columns=['month_of_year'], prefix='m')</code>	Replace a column of categories with one 0/1 column per category. Twelve months become twelve columns <code>m_1</code> ... <code>m_12</code> , and in each row exactly one of them is 1 (one-hot).	Cell 5
<code>dataset.dropna()</code>	Remove every row that contains at least one NaN .	Cell 5
<code>.reset_index(drop=True)</code>	Renumber the rows 0, 1, 2, ... after some rows were removed.	Cell 5
<code>dataset.shape</code>	The size of the table as (rows , columns).	Cells 5, 7
<code>dataset.iloc[:split]</code> and <code>dataset.iloc[split:]</code>	Select rows by <i>position</i> . <code>[:split]</code> means "all rows before position split "; <code>[split:]</code> means "all rows from position split to the end".	Cell 6
<code>.astype('float32')</code>	Store the numbers in the 32-bit decimal format that Keras prefers.	Cell 7
<code>.values</code>	The plain array of numbers inside a Series, without the row numbers.	Cell 13
<code>np.mean(x)</code> and <code>np.abs(x)</code>	The average of the numbers in x ; the absolute value of each number in x (the sign removed).	Cell 13
<code>.round(1)</code>	Round to one decimal place.	Cells 9, 15
<code>.flatten()</code>	Turn a one-column array of 46 rows into a plain list of 46 numbers.	Cell 13
<code>plt.figure(figsize=(10, 3.2))</code> , <code>plt.plot(x, y)</code> , <code>plt.xlabel(...)</code> , <code>plt.legend()</code> , <code>plt.show()</code>	Start a figure of a given size in inches; draw a line through the points (x, y); label an axis; add the legend; display the figure.	Cells 4, 12, 14

C3. Keras (the neural-network library)

Keras is the part of TensorFlow that builds and trains networks. Six words cover everything the notebook does with it.

Code	What it means	Where you meet it
<code>layers.Normalization(axis=-1)</code> and <code>.adapt(...)</code>	A layer that rescales every input column to mean 0 and standard deviation 1. adapt computes the means and standard deviations from the rows you give it.	Cells 8, 9
<code>keras.Sequential(...)</code>	A network built as a stack of layers, listed top to bottom in the order the data flows through them. This is the architecture picture from the lecture, written as a list.	Cell 10
<code>layers.Dense(64, activation='relu')</code>	A layer of 64 perceptrons. Each unit computes $g(w^T x + b)$ from all of the layer's inputs, with its own weights w and bias b ; activation is the function g . A Dense layer with no activation computes the plain weighted sum $w^T x + b$.	Cell 10

Code	What it means	Where you meet it
<code>model.compile(optimizer=r..., loss=...)</code>	Choose the objective function $J(W)$ that training will minimize (loss) and the version of gradient descent that will minimize it (optimizer). <code>keras.optimizers.Adam(0.01)</code> is gradient descent with an adaptive step; 0.01 is the learning rate η .	Cell 10
<code>model.summary()</code>	Print the layers with the number of weights in each.	Cell 10
<code>model.fit(X, y, epochs=..., validation_split=..., callbacks=...)</code>	Train: run the training loop (forward pass, loss, backpropagation, weight update) over the rows of X with answers y . One epoch is one pass over all the training rows. validation_split=0.2 sets aside the last 20 percent of the rows for scoring only. callbacks are rules that run during training, such as early stopping. It returns a <i>history</i> of the losses per epoch.	Cells 11, 18, 19
<code>keras.callbacks.EarlyStopping(monitor='val_loss', patience=40, restore_best_weights=True)</code>	The early-stopping rule from the lecture: watch the validation loss, stop when it has not improved for 40 epochs, and keep the weights from the best epoch.	Cells 11, 18
<code>model.predict(X)</code>	The forward pass: compute the network's output for each row of X . This is the calculation you did by hand in the lecture, done for 46 rows at once.	Cells 13, 18

Part D — The Notebook, Cell by Cell

This part walks through all twenty cells in order. For each cell you will find the code as it appears in the notebook, an explanation of what every line does, what you should see when the cell has run, and, where the class raises a question, the question and its answer. The six cells with blanks are marked YOUR TURN; the hints for their blanks are here, the answers are not.

Stage 1 — Load and Look (Cells 1 to 4)

Cell 1 — Install

Cell 1

```
# Colab already ships TensorFlow, pandas, numpy, matplotlib – nothing to install.
import sys; print(sys.version)
```

The first line is a comment. The second line loads Python's own `sys` module and prints the version of Python that the runtime is using; you should see something that begins with `3.` followed by more numbers and details. The cell exists only to prove that the runtime is alive and that cells run. Nothing is installed, because Colab already provides every library we need.

Cell 2 — Imports

Cell 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print('TensorFlow', tf.__version__)
np.random.seed(510); tf.random.set_seed(510)
```

This cell loads the toolkit for the whole lab and gives each library its customary nickname:

- `numpy` (nickname `np`) works with arrays of numbers: means, absolute values, ranges.
- `pandas` (nickname `pd`) works with tables: reading the CSV file, adding columns, shifting, selecting rows.
- `matplotlib.pyplot` (nickname `plt`) draws plots.
- `tensorflow` (nickname `tf`) is the numerical engine, and `keras` is the part of it that builds and trains neural networks; `layers` holds the building blocks (Dense, Normalization).

The `print` line shows the TensorFlow version; expect `TensorFlow 2.` followed by more digits. The last line sets the *random seeds*. Training a network uses random numbers twice: to choose the initial weights, and to shuffle the rows into mini-batches. A seed fixes the sequence of random numbers, so that running the notebook twice on the same machine gives the same result. It does not make results identical across different machines or software versions, which is one reason your numbers will differ from your neighbor's.

Cell 3 — Load data

Cell 3

```
import os
if not os.path.exists('demand_monthly.csv'):
    from google.colab import files
    print('Please upload demand_monthly.csv'); files.upload()

raw = pd.read_csv('demand_monthly.csv')
raw.tail()
```

The first four lines check whether a file named `demand_monthly.csv` already exists in the runtime. If it does not, they print a request and open Colab's upload button (Choose Files); click it and pick the file on your computer. The upload takes a second. If the file is already there (for example because you ran this cell before), the upload step is skipped.

`pd.read_csv` then reads the file into a DataFrame named `raw`: a table with 240 rows, numbered 0 to 239, and the two columns `month` and `demand`. The last line, `raw.tail()`, displays the last five rows of the table. It has no `print` because a bare expression on the last line of a cell is displayed automatically.

Expected output. A small table with five rows numbered 235 to 239, ending with `2024-11 465` and `2024-12 486`. The numbers at the far left are the row numbers, which start at 0, so the row numbered 239 is the 240th and last month.

Common mistake. `FileNotFoundError` means the file was not uploaded, or its name is different (for example `demand_monthly (1).csv` after a second download). Rename it to exactly `demand_monthly.csv`, or upload it again.

Cell 4 — Clean and look

Cell 4

```
dataset = raw.copy()
print(dataset.isna().sum())          # no missing values expected
dataset['t'] = np.arange(len(dataset))
dataset['month_of_year'] = (dataset['t'] % 12) + 1

plt.figure(figsize=(10,3.2))
plt.plot(dataset['t'], dataset['demand'], lw=1.2)
plt.title('Monthly demand – trend + seasonality + noise')
plt.xlabel('month index t'); plt.ylabel('units'); plt.show()
```

Line by line:

- `dataset = raw.copy()` makes a working copy of the raw table, so that `raw` itself stays untouched. Everything from here on is done to `dataset`.
- `print(dataset.isna().sum())` counts the missing values in each column and prints the counts. Both should be 0. Looking for missing values is the first thing to do with any data file, even when you expect none.
- `dataset['t'] = np.arange(len(dataset))` adds a column `t` holding the month index 0, 1, 2, ..., 239. This is the "t" of the lecture: a simple counter of months. It is convenient for plotting, and the split in Stage 3 uses it to confirm that the test months come after the training months.

- `dataset['month_of_year'] = (dataset['t'] % 12) + 1` adds the calendar month as a number from 1 (January) to 12 (December). It works because the series starts in January: for $t = 0$ the remainder is 0 and the month is 1; for $t = 11$ the month is 12; for $t = 12$ the remainder is 0 again and the month is 1, which is January 2006. This column is only a stepping stone; in Cell 5 it is turned into twelve switches.
- The remaining lines draw the series: a figure 10 inches by 3.2 inches, a line through the points (t , demand), a title, axis labels, and `plt.show()` to display it.

Stop here and look at the plot, and say the three ingredients out loud: trend, seasonality, noise. If you cannot see a pattern in the data, no model will find one either.

Question. Where are the trend, the seasonality, and the noise in this plot?

Answer. The trend is the steady rise of the whole curve, from below 200 units at the left to almost 500 units at the right; it is the level drifting upward at about ten units per year. The seasonality is the repeating shape inside each year: a sharp peak at the end of every year (November and December) followed by a deep dip at the start of the next year (January and February), the same shape twenty times. The noise is everything that does not repeat: the small irregular wiggles of a few units that make one year differ slightly from the next. These are the same three ingredients that triple exponential smoothing tracks with a level, a trend, and seasonal factors.

Checkpoint. After Cell 4 the table `dataset` has 240 rows and 4 columns: `month`, `demand`, `t`, and `month_of_year`. Cell 4 does not print the shape, but you can see it for yourself: click "+ Code" to add a new cell, type `dataset.shape`, and run it; it prints `(240, 4)`. The plot should show the rising, saw-toothed series with its year-end peaks.

Stage 2 — Build the Training Table (Cell 5, YOUR TURN)

This is the cell that turns forecasting into learning. Before Cell 5 we have one time series. After Cell 5 we have a table of *examples*, where each row is one question-and-answer pair: the question is twenty-four numbers (the twelve previous months and the twelve month switches) and the answer is that month's demand. A network learns from examples, so it needs this table, not the series.

Cell 5 — YOUR TURN (two blanks)

```
# one-hot month (12 columns: m_1 ... m_12)
dataset = pd.get_dummies(dataset, columns=['month_of_year'], prefix='m', dtype='float32')

# lag features: lag_k = demand k months earlier (the window, slid over the history)
for k in range(1, ):
    dataset[f'lag_{k}'] = dataset['demand'].shift(k)

dataset['label_next'] = dataset[''] # the answer key: THIS month's demand, predicted
from the previous 12
dataset = dataset.dropna().reset_index(drop=True) # the first 12 rows have no full window
print(dataset.shape)
dataset.head(2)
```

The month switches (line 2). `pd.get_dummies` takes the column `month_of_year`, which holds a number from 1 to 12, and replaces it with twelve new columns named `m_1`, `m_2`, ..., `m_12`. In each row, the column of

that row's month holds 1.0 and the other eleven hold 0.0. Exactly one switch is on per row, which is why this is called *one-hot* encoding. The `prefix='m'` argument chooses the names, and `dtype='float32'` stores the switches as decimal numbers rather than as True/False, because the network needs numbers.

The lag columns (lines 4 to 6). A lag is a value from an earlier month. `dataset['demand'].shift(k)` is the demand column moved *down* by *k* rows, so that in every row the shifted column holds the demand from *k* months earlier. The loop runs this once for each *k* and stores the result as a new column named `lag_k` (the f-string `f'lag_{k}'` builds the name `lag_1`, `lag_2`, and so on). Together, the lag columns are the sliding window from the lecture diagram: instead of sliding a window along the series one position at a time, the twelve shifts produce every position of the window at once, one per row. The first *k* rows of `lag_k` have no earlier month to copy from, so they are empty (`NaN`).

The label (line 8). The label is the answer that goes with each row's question: the thing the network is asked to predict. For the row of a given month, the question is the twelve months *before* it plus its month switches, and the answer is that month's own demand. From the window's point of view, that month is the "next month" after the window, which is why the column is named `label_next` and why the lecture writes the target as $\hat{A}(t+1)$. Both phrasings describe the same row.

Dropping the incomplete rows (line 9). The twelve months of 2005 have no full year before them, so at least one of their lag cells is `NaN`. `dropna()` removes every row containing a `NaN`, and `reset_index(drop=True)` renumbers the remaining rows from 0. **The print (line 10)** shows the size of the finished table as `(rows, columns)`, and the last line displays its first two rows.

Blank 1. How many lag months should the window hold? The loop runs `for k in range(1, N)`, and `range(1, N)` produces 1, 2, ..., *N*-1; it stops *before* *N*. Choose *N* so that the window holds the twelve months of the lecture diagram. If you write 12 you get only eleven lags, and the CHECK cell will tell you so.

Blank 2. Which column is the label, the thing we are predicting? The answer is a column name, written inside the quotation marks exactly as the surrounding code does.

Expected output. `(228, 28)` and then the first two rows of the table. Row 0 is 2006-01: its `lag_1` is 251.0 (the demand of December 2005), its `lag_12` is 169.0 (January 2005), its `m_1` switch is 1.0 and the other switches are 0.0, and its `label_next` is 177, the demand of January 2006 itself. Read that row slowly: it is one complete question-and-answer pair.

Checkpoint. The CHECK cell prints "✓ Cell 5 check passed: 228 rows (240 minus the first 12, which have no full window); 24 inputs = 12 lags + 12 month switches; 1 label". It tests three things: that there are exactly twelve lag columns, that the label equals the demand column, and that the shape is (228, 28).

Question. Why does the table have 228 rows when the series has 240 months?

Answer. Each row needs a full window of twelve earlier months. The twelve months of 2005 do not have that: January 2005 has no earlier month at all, and December 2005 has only eleven. Their lag cells are therefore empty, and `dropna()` removes all twelve rows. 240 minus 12 is 228. The first usable row is January 2006, whose window is exactly the twelve months of 2005.

Question. Why does the table have 28 columns, and which of them does the network actually see?

Answer. The 28 columns are `month`, `demand`, `t`, the twelve switches `m_1` to `m_12`, the twelve lags `lag_1` to `lag_12`, and `label_next`: $1 + 1 + 1 + 12 + 12 + 1 = 28$. Only the 24 in the middle, the twelve lags and the twelve switches, are inputs to the network; Cell 7 selects exactly those. `month` is text, `demand` and `label_next` are the answer (they are the same numbers), and `t` is deliberately not given to the network, so the network has no clock: it must infer the level from the lags, not from a counter.

Question. Why is the month represented as twelve switches instead of the single number 1 to 12?

Answer. Because the month is a category, not a quantity. If the network received the number 12 for December and 1 for January, it would be told that December is "twelve times January" and that November and December are "closer" than December and January, which is false: December and January are neighbors. With twelve switches, each month gets its own input and its own weights, so the network can learn that November and December are high and January and February are low without any false arithmetic between months. This is exactly how a categorical predictor enters a regression, as 0/1 dummy variables.

Question. The lecture diagram shows a window sliding along the series. Where is the sliding in this code?

Answer. In the twelve `shift` calls. Look at any two consecutive rows of the finished table. Row 0 (January 2006) holds the demands of January to December 2005 in `lag_12` to `lag_1`; row 1 (February 2006) holds February 2005 to January 2006. The window has moved forward by one month, and its contents have moved along by one column. Every row of the table is one position of the window, so building the twelve shifted columns is the same thing as sliding the window over the entire history and writing down what it sees at every stop.

Stage 3 — Split by Time (Cells 6 and 7)

Cell 6 — YOUR TURN (three blanks)

Cell 6 — YOUR TURN (three blanks)

```
split = int(len(dataset) * )
train_dataset = dataset.iloc[].copy()      # the past
test_dataset = dataset.iloc[].copy()      # the future
print(len(train_dataset), 'train months |', len(test_dataset), 'test months')
```

`len(dataset)` is 228, the number of rows. Multiplying it by the training fraction and converting with `int` (which drops the decimals) gives the row position `split` at which the table is cut. `dataset.iloc[:split]` selects every row *before* that position, and `dataset.iloc[split:]` selects every row *from* that position to the end. The `.copy()` makes each part an independent table. The print shows how many months went to each side.

Blank 1. The training fraction. We keep the most recent 20 percent of the months for testing, so the fraction that goes to training is the rest, written as a decimal.

Blanks 2 and 3: Which slice is the past and which is the future? Rows before the cut are earlier in time. Put the past in `train_dataset` and the future in `test_dataset`.

Expected output. 182 train months | 46 test months. With 228 rows, the training fraction gives 182.4, which `int` turns into 182; the remaining 46 rows are the test set, roughly the last four years (March 2021 to December 2024).

Checkpoint. The CHECK cell confirms the counts 182 and 46 and, more importantly, that the smallest `t` in the test set is larger than the largest `t` in the training set, that is, that the test set comes *after* the training set. If you swapped the slices, it says "the test set must come AFTER the training set".

Question. For the regression examples we could pick the test points at random. Why not split this table at random too?

Answer. Because a forecast is a statement about the future, and a random split would let the model see the future's neighbors. Suppose March 2015 were a randomly chosen test month. The training set would then contain February 2015 and April 2015, and also March 2014 and March 2016, so the model would be tested on a month whose surroundings it has already studied. That is interpolation between known months, an easy task, and it would report a flattering accuracy that planning could never achieve in practice. Holding out the *last* 46 months tests what planning really needs: predicting months the model has never seen, including a demand level higher than anything in the training years.

Question. Why the last 20 percent, and not, say, the last six months?

Answer. Forty-six months is almost four years, so the test period contains several complete seasonal cycles. A MAD averaged over that many months is a fair measure; a MAD over six months would depend heavily on which six months they happened to be. At the same time, 80 percent leaves 182 months of history to learn from. The 80/20 division is a convention, not a law; what matters is that the test block is the most recent one and long enough to be representative.

Cell 7 — Split features and labels

Cell 7

```
feature_cols = [f'lag_{k}' for k in range(1,13)] + [c for c in dataset.columns if
c.startswith('m_')]

train_features = train_dataset[feature_cols].astype('float32')
test_features  = test_dataset[feature_cols].astype('float32')
train_labels   = train_dataset['label_next'].astype('float32')
test_labels    = test_dataset['label_next'].astype('float32')
print(train_features.shape, test_features.shape)    # 24 inputs per row
```

The first line builds the list of input column names. The first list comprehension produces the twelve names `lag_1` to `lag_12`; the second picks out every column name that starts with `m_`, which are the twelve switches; the `+` joins the two lists into one list of 24 names. The next four lines cut each of the two tables into its inputs (the *features*, the 24 columns) and its answers (the *labels*, the single `label_next` column), converting

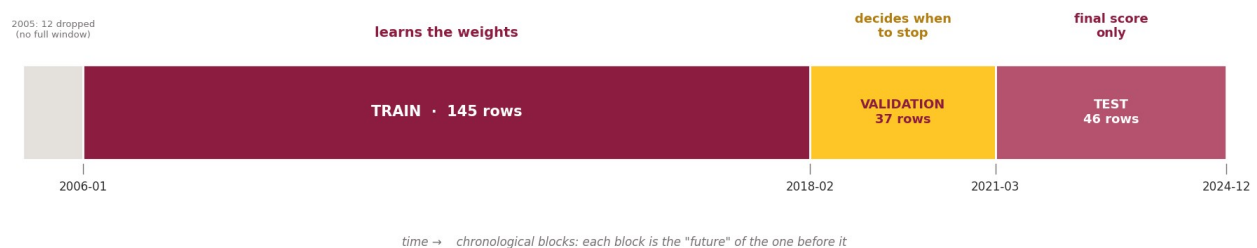
everything to the `float32` number format that Keras prefers. From here on, `train_features` and `train_labels` are the question-and-answer pairs the network learns from, and `test_features` and `test_labels` are the ones it is judged on.

Expected output. (182, 24) (46, 24): 182 training rows and 46 test rows, each with 24 input numbers.

Lecture link. In the regression notation of Part 5, the training features are the K-by-m matrix of inputs with K = 182 rows and m = 24 inputs, and the training labels are the vector of K outputs.

Three Splits, Three Jobs

Before building the network, be precise about where the 240 months went, because there are *three* blocks, not two, and each block has exactly one job.



Where the 240 months go: 12 dropped, then three chronological blocks. Training learns, validation decides when to stop, test judges.

- **Dropped (12 months, all of 2005).** These rows have no full window behind them and were removed in Cell 5.
- **Training (145 rows, January 2006 to January 2018).** Gradient descent sees only these rows. Every weight update is computed from them; this is the answer key the network studies.
- **Validation (37 rows, February 2018 to February 2021).** When Cell 11 calls `model.fit` with `validation_split=0.2`, Keras takes the *last* 20 percent of the 182 training rows and sets them aside: 37 rows that are never used for updating the weights but are scored after every epoch. That score is the validation loss, the gold curve in Cell 12, and early stopping watches it to decide when to stop training and which weights to keep.
- **Test (46 rows, March 2021 to December 2024).** The future. It is touched exactly once, in Cell 13, to compute the scorecard. It never influences the weights or the stopping rule.

Notice that the blocks are chronological: validation is the future of training, and test is the future of both. Training learns, validation decides when to stop, test judges.

Question. We already have a test set. Why is a validation set needed as well?

Answer. Because deciding when to stop training, and which epoch's weights to keep, is itself a choice made from data. If that choice were made by looking at the test months, the test months would have shaped the model, and the test score would no longer be an honest measure of forecasting a future the model never saw; it would be flattery. So a slice is carved off the *training* block to make the stopping decision, and the test block stays untouched until the very last step. In short: training data is for fitting the weights, validation data is for choosing how long to fit, and test data is only for the final grade.

Question. Why is the validation block the last part of the training block rather than a random sample of it?

Answer. For the same reason the test block is the last part of the series. Keras always takes the last fraction of the rows for validation, and for time series that is exactly right: the validation months are the near future of the months the network trains on, so the validation loss imitates the situation the network will face on the real test months. A random validation sample would again let the network be judged on months whose neighbors it had trained on.

Stage 4 — Normalize and Build the Network (Cells 8 to 10)

Cell 8 — Normalization layer

Cell 8

```
normalizer = layers.Normalization(axis=-1)
normalizer.adapt(np.array(train_features))
```

The first line creates a normalization layer: a layer that, for every one of the 24 input columns, subtracts that column's mean and divides by its standard deviation, so that each input arrives at the network with mean 0 and standard deviation 1 (**axis=-1** says "do this column by column"). The second line, **adapt**, computes those 24 means and 24 standard deviations from the *training* rows. Those statistics are stored inside the layer and are never changed by training; that is why **model.summary()** in Cell 10 lists them as 49 non-trainable numbers (24 means, 24 variances, and a count).

This is the grown-up version of the "scaled by 100" step in the hand-worked forward-pass example. The lags are numbers in the hundreds while the switches are 0 or 1; without rescaling, the lags would dominate every weighted sum, and a learning rate that suits one input would be far too large or far too small for another, so gradient descent would crawl or bounce.

Question. Why are the means and standard deviations computed from the training rows only, and not from the whole table?

Answer. Because nothing about the test months may leak into the model, not even their averages. The test months sit at a higher level than the training months. If the normalizer had seen them, the model would already "know" that the future is higher, which is information no forecaster has at the time of forecasting. Computing the statistics from the training rows keeps the test an honest test. The same rule applies to every preprocessing step in forecasting: fit it on the past, apply it to the future.

Cell 9 — Check normalization on one sample

Cell 9

```
first = np.array(train_features[:1])
print('raw   :', first[0][:6].round(1), '...')
print('normed:', normalizer(first).numpy()[0][:6].round(2), '...')
```

This cell takes the first training row (January 2006), prints its first six inputs as they are in the table, and then prints the same six inputs after they have passed through the normalizer. The first six inputs are `lag_1` to `lag_6`, that is, December 2005 back to July 2005.

Expected output. `raw : [251. 250. 211. 205. 217. 224.] ...` and `normed: [-0.67 -0.68 -1.38 -1.48 -1.25 -1.11] ....` A normalized value of `-0.67` means "0.67 standard deviations below this column's training mean".

Question. Why are all six normalized values negative?

Answer. Because this is the first row of the table, from early 2006, when demand was around 200 to 250 units, while the training means of the lag columns are around 285 units (the average over the training years, 2006 to early 2021). Every early lag therefore sits below its column's mean and becomes negative after normalization. Rows from the late 2010s, when demand was above 300, show positive values. The normalizer has not changed the information in the row; it has only expressed each number relative to what is typical for its column.

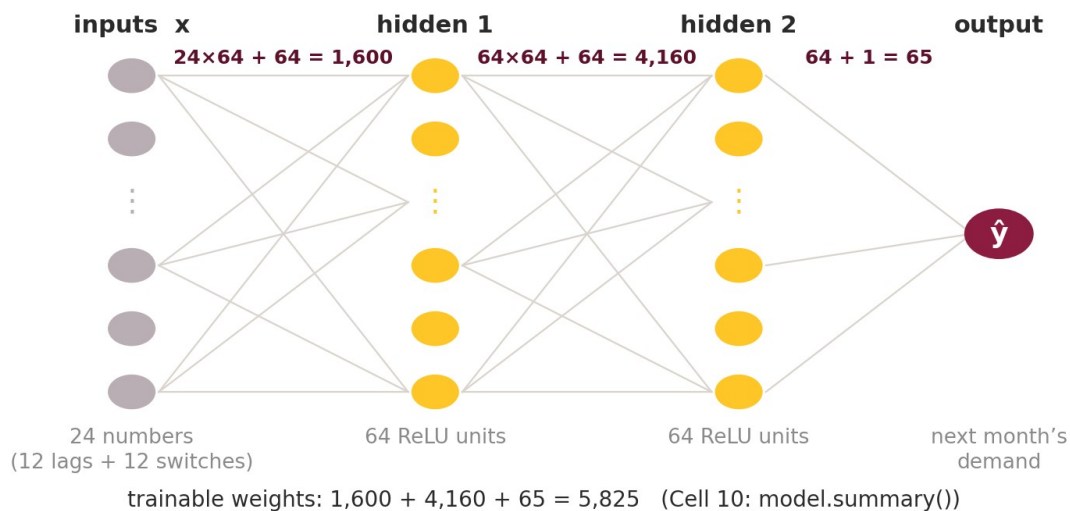
Cell 10 — YOUR TURN: build and compile the model (four blanks; the first two ask the same question)

Cell 10 — YOUR TURN (four blanks)

```
model = keras.Sequential([
    normalizer,
    layers.Dense(64, activation='____'),
    layers.Dense(64, activation='____'),
    layers.Dense(____),
])
model.compile(optimizer=keras.optimizers.Adam(0.01), loss='____')
model.build(input_shape=(None, 24)) # tell Keras the input size, so summary() can count the
weights
model.summary()
```

The architecture (lines 1 to 6). `keras.Sequential([...])` builds a network as a stack of layers, listed in the order the data flows through them. The data enters the normalizer first, then passes through two hidden layers, then reaches the output layer:

- `layers.Dense(64, activation='____')` is a layer of 64 perceptrons, the perceptron from the lecture slides repeated 64 times. Each unit receives all of the layer's inputs (24 numbers for the first hidden layer, 64 for the second), forms its own weighted sum $w^T x + b$, and applies the activation function `g`. The first two blanks ask for `g`.
- `layers.Dense(____)` is the output layer. It has no `activation` argument, so it computes a plain weighted sum of the 64 numbers coming out of the second hidden layer. The third blank asks how many units this layer needs.



The network of Cell 10: 24 inputs, two hidden layers of 64 ReLU units, one output unit. The number of weights in each layer is written above it.

Compile (line 7). `model.compile` fixes two decisions before training starts. The `optimizer` is the version of gradient descent that will be used: `Adam` is gradient descent with an adaptive step size, and the number 0.01 is the learning rate η from the lecture, the basic step size. The `loss` is the objective function $J(W)$ that gradient descent minimizes: a formula that measures how wrong the network's outputs are on the training rows. The fourth blank asks for it.

Build and summary (lines 8 and 9). `model.build(input_shape=(None, 24))` tells Keras that each input row has 24 numbers (`None` stands for "any number of rows"), so that the weights can be created. `model.summary()` then prints a table with one line per layer and the number of weights in each.

Blanks 1 and 2: Which activation function g did the hand-worked forward-pass example use? It is the one that lets a unit switch off (output 0) for some inputs and pass the weighted sum through for others; Keras spells its name in lowercase letters, inside quotation marks.

Blank 3. How many output units does a network need when it predicts one quantity, next month's demand? A number, without quotation marks.

Blank 4: We want the objective $J(W)$ to be the forecaster's MAD from Lecture 1, Part 2, the mean of the absolute errors. Keras calls that loss '`mean_absolute_error`'; the alternative, which would make the objective the mean squared error (MSD in Part 2), is '`mean_squared_error`'. Choose the one that equals MAD.

Expected output. A table listing `normalization` with 49 parameters, `dense` with 1,600, `dense_1` with 4,160, and `dense_2` with 65, followed by "Total params: 5,874", "Trainable params: 5,825", and "Non-trainable params: 49". The column "Output Shape" shows `(None, 64)` for the hidden layers, meaning "any number of rows, 64 numbers each", and `(None, 1)` for the output.

Checkpoint. The CHECK cell confirms that the hidden layers use the right activation, that the output layer has the right number of units, that the total number of parameters is $5,825 + 49$, and that the loss is the absolute-error one. It prints "✓ Cell 10 check passed: $24 \rightarrow 64 \rightarrow 64 \rightarrow 1$, MAE loss".

Question. Where do the numbers 1,600, 4,160, and 65 come from? Count the weights.

Answer. A Dense layer with n inputs and u units has $n \times u$ weights (one per input per unit) plus u biases (one per unit). The first hidden layer has 24 inputs and 64 units: $24 \times 64 + 64 = 1,536 + 64 = 1,600$. The second hidden layer has 64 inputs and 64 units: $64 \times 64 + 64 = 4,096 + 64 = 4,160$. The output layer has 64 inputs and 1 unit: $64 \times 1 + 1 = 65$. The total is $1,600 + 4,160 + 65 = 5,825$ trainable weights. The 49 numbers in the normalizer are not weights; they are the means and variances measured in Cell 8, and they are never changed by training.

Question. Why does the output layer have no activation function?

Answer. Because the output is a demand forecast, a plain quantity that may take any value in the hundreds. An activation such as the sigmoid would squash the output into a fixed range, and even ReLU would clip anything negative, which is unnecessary. So the output unit is a plain weighted sum of the 64 numbers produced by the second hidden layer: in effect, a linear regression on top of 64 learned features. The hidden layers are where the non-linearity lives; the output layer just combines what they found.

Question. Why do the hidden layers need an activation function at all?

Answer. Without one, each layer would be a linear map, and a stack of linear maps is still a linear map: the whole network would collapse into a single linear regression on the 24 inputs, no matter how many layers it had. The activation function is what makes the network non-linear. With ReLU, $g(z) = \max(0, z)$, each unit is switched off in one region of the input space and passes its weighted sum through in the other, so different units are active for different kinds of months, and the network can bend its response instead of drawing one straight line. That is the same mechanism you saw in the hand-worked example, where one unit switched off for the given input.

Question. Why 64 units, and why two hidden layers?

Answer. This is a design choice, not a law. More units and more layers make the function more flexible, but they also add weights that must be fitted from only 145 training rows, and more weights means more opportunity to memorize noise. Two layers of 64 are small enough to train in seconds and flexible enough to capture a trend and a seasonal profile. You are welcome to try other sizes in the final project.

Question. Why is the loss MAD (mean absolute error) rather than MSD (mean squared error)?

Answer. Both are legitimate objectives, and Part 2 of the lecture showed that they can rank forecasts differently, because squaring punishes large errors much more heavily than small ones. We choose the absolute error because MAD is the scorecard we report at the end; making the objective the same as the score means gradient descent is aiming at exactly the number we care about. If a planner were much more afraid of large misses than of small ones, mean squared error would be the natural choice instead.

Stage 5 — Train (Cells 11 and 12)

Cell 11 — Train the model

Cell 11

```
# Early stopping (Regularization 2 from the lecture): stop when validation loss
# has not improved for 40 epochs, and keep the best weights seen.
early_stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=40,
                                           restore_best_weights=True)

history = model.fit(
    train_features, train_labels,
    epochs=300, validation_split=0.2,
    callbacks=[early_stop], verbose=0)

print('epochs run:', len(history.history['loss']),
      '| best validation loss (MAE):', round(min(history.history['val_loss']), 2))
```

This cell is gradient descent, live. It runs for twenty to forty seconds; the spinning icon at the left of the cell shows that it is working.

The early-stopping rule (lines 3 and 4). A *callback* is a rule that Keras applies during training. `EarlyStopping` watches the validation loss (`monitor='val_loss'`) after every epoch. `patience=40` means: if the validation loss has not improved for 40 epochs in a row, stop. `restore_best_weights=True` means: when training stops, go back to the weights from the epoch with the lowest validation loss, rather than keeping the last ones. This is the early-stopping idea from the lecture's regularization slides, exactly as stated there.

Training (lines 6 to 9). `model.fit` runs the training loop. Its arguments:

- `train_features, train_labels`: the 182 question-and-answer pairs.
- `validation_split=0.2`: set aside the last 37 of those rows for scoring only (the validation block of the figure in the previous section). The weights are updated using the other 145 rows.
- `epochs=300`: an *epoch* is one pass over all 145 training rows. This is an upper limit; early stopping normally ends training well before it.
- Within each epoch, Keras processes the rows in *mini-batches* of 32 (its default size): 32, 32, 32, 32, and the remaining 17. For each mini-batch it runs the forward pass, computes the loss, computes the gradient of the loss with respect to all 5,825 weights by backpropagation, and takes one Adam step. So one epoch is five weight updates, and a run of 80 epochs is about 400 updates.
- `callbacks=[early_stop]`: apply the early-stopping rule.
- `verbose=0`: do not print a progress line for every epoch.

`model.fit` returns a *history*, a record of the training loss and the validation loss after every epoch. The `print` line uses it to report how many epochs were actually run (the length of the loss record) and the best validation loss reached.

Expected output. One line such as `epochs run: 79 | best validation loss (MAE): 8.34`. Your two numbers will differ; anything from about 60 to 150 epochs and a best validation loss of roughly 7 to 12 units is typical. The number of epochs is always the best epoch plus 40, because training continued for 40 more epochs looking for an improvement that did not come.

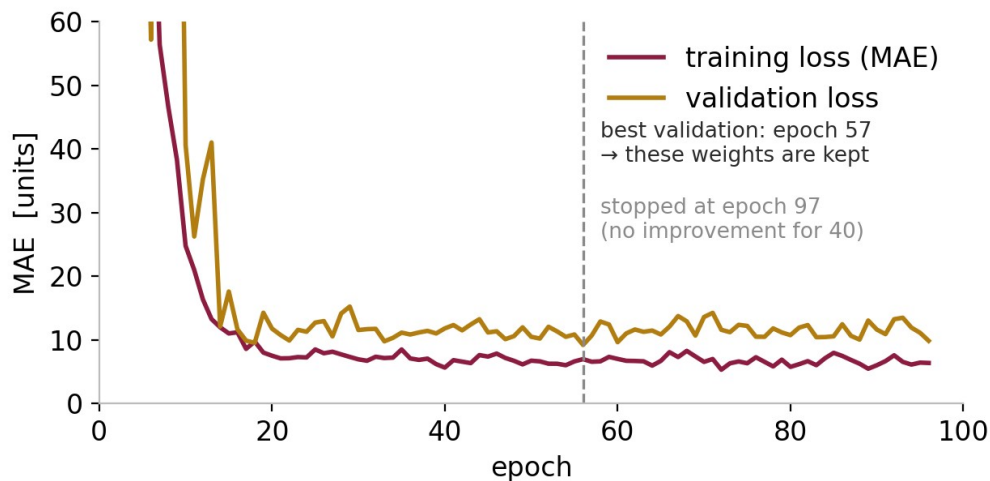
Lecture link. The loss surface, the gradient, and the step of size η are Lecture 0, Section 5. Backpropagation, mini-batches, epochs, and early stopping are the neural-network section of Lecture 1.

Cell 12 — Plot the training history

Cell 12

```
plt.figure(figsize=(7,3))
plt.plot(history.history['loss'], label='train loss (MAE)')
plt.plot(history.history['val_loss'], label='validation loss')
plt.xlabel('epoch'); plt.ylabel('MAE [units]'); plt.legend(); plt.show()
```

This cell draws the *learning curve*: the training loss and the validation loss after every epoch, both in units of demand. The figure below is a typical run, with the best epoch and the stopping epoch marked; your own plot will not have those markers, but the shape should be the same.



A typical learning curve. Both losses fall quickly and flatten. The validation curve stays above the training curve, its lowest point is the best epoch, and training stops 40 epochs later.

How to read it: in the first ten to twenty epochs both curves fall steeply, from errors of hundreds of units down to about ten; that is gradient descent walking down the loss surface. Then both flatten. The training loss settles lower (around 5 to 7 units) than the validation loss (around 8 to 12 units). Somewhere on the flat part the validation loss reaches its minimum; that epoch's weights are the ones that early stopping restored. After it, the curves wander without improving, and 40 epochs later training stops.

Question. Why is the validation loss higher than the training loss?

Answer. The training loss is measured on the very rows whose answers the network used to set its weights, so part of that fit is memorization of those particular rows, including their noise. The validation rows were never used for an update, so the validation loss measures how well the learned pattern carries over to months the network did not study. The gap between the two curves is the size of that memorization. Some gap is normal; a gap that keeps growing is overfitting.

Question. What would overfitting look like on this plot, and what does early stopping do about it?

Answer. Overfitting shows up as the two curves parting company: the training loss keeps creeping down because the network keeps fitting the noise in its 145 rows, while the validation loss stops improving and drifts upward, because the noise it is now fitting does not exist in other months. Early stopping is the remedy from the regularization slides: it notices that the validation loss has not improved for 40 epochs, stops, and restores the weights from the epoch where the validation loss was lowest, which is the point of best generalization rather than best memorization.

Question. Why not simply train for all 300 epochs and keep the final weights?

Answer. Because the final weights are not the best weights. Past the best epoch, additional updates mostly fit noise, so the final weights would typically forecast the test months worse than the weights from the best epoch, and how much worse would depend on the accident of where training happened to end. Early stopping with restored weights removes that arbitrariness. You can see what happens without it by switching early stopping off in the experiment helper of Cell 19.

Question. My neighbor's curve and numbers are different from mine. Is something wrong?

Answer. No. The initial weights are random and the rows are shuffled into mini-batches in a random order, so every training run walks a slightly different path down the loss surface and stops at a slightly different point. Seeds make a run repeatable on one machine, but not identical across machines. The shape of the curve, the rough level of the losses, and the ranking of the forecasts on the scorecard should agree; the decimals will not.

Stage 6 — Score Like a Forecaster (Cells 13 to 15)

Cell 13 — YOUR TURN: score the network (three blanks)

Cell 13 — YOUR TURN (three blanks)

```
pred = model.predict(test_features, verbose=0).flatten()
actual = test_labels.values

def mad(a, f): return       
def mape(a, f): return np.mean(np.abs((f - a) / a)) * 100

# Baselines: naive (copy last month) and seasonal-naive (copy the same month last year)
naive = test_dataset['      '].values
snaive = test_dataset['      '].values

print(f"Neural network : MAD = {mad(actual, pred):6.2f}    MAPE = {mape(actual, pred):5.2f}%")
print(f"Naive (lag-1) : MAD = {mad(actual, naive):6.2f}    MAPE = {mape(actual, naive):5.2f}%")
print(f"Seasonal naive : MAD = {mad(actual, snaive):6.2f}    MAPE = {mape(actual, snaive):5.2f}%")
```

The forecasts (lines 1 and 2). `model.predict(test_features)` runs the forward pass on the 46 test rows, the calculation you did by hand in the lecture, and returns 46 forecasts; `.flatten()` turns the 46-by-1 result

into a plain list of 46 numbers named `pred.actual` holds the 46 true demands of the test months. This is the only time the test set is used.

The two accuracy measures (lines 4 and 5). `def mad(a, f)` defines a function with two inputs, the actual values `a` and the forecasts `f`, and `return` gives back its result. MAD is the mean absolute deviation from Lecture 1, Part 2: the average, over the test months, of the absolute error, that is, of $|\text{forecast} - \text{actual}|$. The first blank asks you to write that formula in code. `mape` is given: it is the mean absolute *percentage* error, the average of $|\text{forecast} - \text{actual}|$ divided by actual, times 100.

The baselines (lines 8 and 9). A baseline is a forecast that needs no model. The *naïve* forecast for a month is simply the demand of the month before it; the *seasonal naïve* forecast is the demand of the same month one year earlier. Both are already sitting in the training table as lag columns, so each baseline is just a column of `test_dataset`. The second and third blanks ask which lag columns those are.

The scorecard (lines 11 to 13). The three `print` lines use f-strings with formatting codes: `:6.2f` means "a number with two decimals, padded to six characters wide", which keeps the columns aligned.

Blank 1. Write MAD as code: the mean of the absolute value of `f - a`. You have `np.mean(...)` for the mean and `np.abs(...)` for the absolute value; put one inside the other.

Blanks 2 and 3: Which lag column holds last month's demand, and which holds the demand of the same month last year? The column names are text and go inside the quotation marks.

Expected output. Three lines. For the two baselines the numbers are exact, because nothing random is involved: naïve MAD 30.57 and MAPE 8.40%; seasonal naïve MAD 11.20 and MAPE 2.91%. For the network, a MAD between about 12 and 16 units and a MAPE between about 3 and 4.5 percent is typical; your value depends on your training run.

Checkpoint. The CHECK cell tests your `mad` function on a tiny example (actual 1, 2, 3 against forecasts 1, 4, 3 must give 2/3), and then recognizes the two baselines by their scores, about 30.6 and about 11.2. If you picked the wrong lag column, the message says which baseline is wrong. It prints "✓ Cell 13 check passed".

Question. Why compare against baselines at all? Is a MAD of 13 units not simply a fact about the network?

Answer. A MAD of 13 means nothing on its own. It is a good number if every simpler method does much worse, and a poor number if a one-line rule does just as well for free. A forecasting method earns its keep only by beating what you could have done without it, and the baseline must be the strongest free one you can think of. Here the naïve forecast (copy last month) is the weakest sensible baseline and the seasonal naïve (copy the same month last year) is a strong one, because the series is so seasonal. Reporting the network's MAD next to both is what makes the number interpretable.

Question. What do "MAD = 13" and "MAPE = 3.4%" mean in plain words?

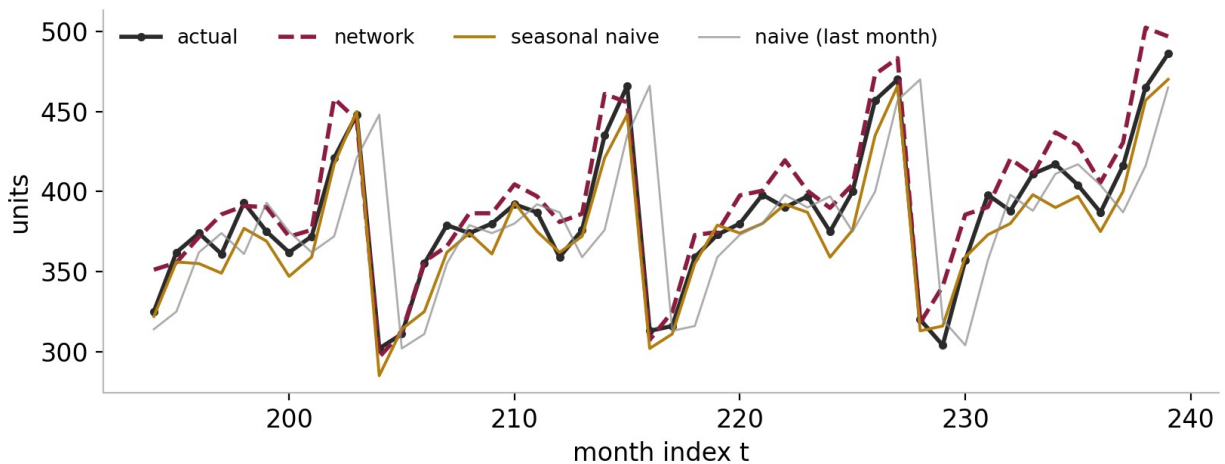
Answer. MAD = 13 means that, averaged over the 46 held-out months, the forecast missed the actual demand by 13 units, counting misses in both directions as positive. MAPE = 3.4% says the same thing as a percentage of the actual demand: on average the forecast was off by 3.4 percent of that month's demand. MAD is in units and is easy to relate to inventory; MAPE is unit-free, so it can be compared across products of very different sizes. Part 2 of the lecture defined both, and both belong on a scorecard.

Cell 14 — The hold-out picture

Cell 14

```
plt.figure(figsize=(10,3.4))
plt.plot(test_dataset['t'], actual, label='actual', lw=1.6)
plt.plot(test_dataset['t'], pred, label='NN forecast', lw=1.6, ls='--')
plt.plot(test_dataset['t'], snaive, label='seasonal naive', lw=1, alpha=0.6)
plt.xlabel('month index t'); plt.ylabel('units'); plt.legend(); plt.title('Hold-out period');
plt.show()
```

This cell draws the 46 test months: the actual demand, the network's forecast (dashed), and the seasonal naive forecast (thin). The figure below is the same picture from a typical run, with the naive forecast added in gray.



The 46 held-out months. Actual demand in black, the network's forecast dashed, the seasonal naive in gold, and the naive forecast in gray.

Question. What should I look for in this plot?

Answer. Three things. First, the seasonal naive line is literally last year's curve: it has exactly the right shape, peak and dip in the right months, but it runs a little low everywhere, because it cannot see that the level has risen by another year of trend. Second, the network's dashed line follows the swings and also the rising level, but it is noisier and sometimes overshoots, especially around the peaks. Third, the gray naive line is the actual curve shifted right by one month: it is late at every turn, which is why its MAD is by far the largest. The plot shows what the scorecard summarizes: the seasonal naive is a very strong competitor on this series.

Cell 15 — A quick prediction table

Cell 15

```
out = test_dataset[['month']].copy()
out['actual'] = actual.astype(int)
out['nn_forecast'] = np.round(pred.astype(float), 1)
out['error'] = (out['nn_forecast'] - out['actual']).round(1)
out.tail(12)
```

This cell builds a small table with the month, the actual demand, the network's forecast rounded to one decimal, and the error defined as forecast minus actual, and it displays the last twelve rows, the twelve months of 2024. A positive error means the network forecast too much (over-forecast); a negative error means it forecast too little. Scan the column: the errors should be a mix of signs and mostly smaller than about 30 units, with the largest ones near the seasonal peaks and dips.

Cell 16 — (Optional) Export the model

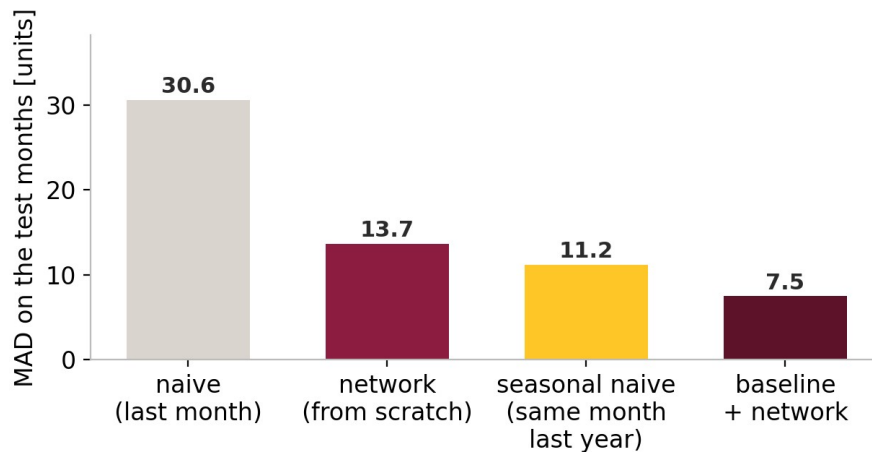
Cell 16

```
try:
    import tf2onnx, onnx
    spec = (tf.TensorSpec((None, len(feature_cols)), tf.float32, name='x'),)
    onnx_model, _ = tf2onnx.convert.from_keras(model, input_signature=spec, opset=13)
    onnx.save(onnx_model, 'demand_nn.onnx')
    print('saved demand_nn.onnx')
except Exception as e:
    print('Export skipped (tf2onnx not installed or not compatible) – optional step, nothing to fix.')
```

A trained network is a product you can hand to other software. This cell tries to save the model in ONNX, a portable file format that many programs can read, so that, for example, a planning system could compute the forecast without Python. The `try/except` structure means that if the conversion library is missing or incompatible, the cell prints a message and moves on instead of stopping. Either outcome is fine; nothing later depends on this cell.

Discussion — Why Is the Free Baseline So Strong? (Cell 17)

Cell 17 is a text cell: it summarizes the scorecard and asks three questions to discuss with your partner. The typical picture is this:



A typical scorecard. The network beats the naive forecast by a factor of two, and the seasonal naive still edges it out. The fourth bar is the challenge of Cell 18.

Two facts, and both are the lesson of the lab. The network beats "copy last month" by a factor of two. And a free, one-line baseline, "copy the same month last year", still does slightly better than the network. Do not be embarrassed by that; be curious about it. The three questions below are the ones on the discussion slide, written out with their answers so that you can check your own reasoning. Make sure you understand the mechanism, not just the conclusion.

Discussion question 1. Why does the seasonal naive forecast do so well on *this* series? Think about how the series was built, and about what "copy the same month last year" silently assumes.

Answer. The series was generated as a smooth straight-line trend, plus a fixed seasonal profile that repeats identically every year, plus white noise. The seasonal naive forecast assumes exactly that structure: it says "this month will look like the same month last year". On this data that assumption is almost perfectly true. The only ways the copy can be wrong are one year of trend, about 10 units, and the difference between this year's noise and last year's noise, a few units each. So the seasonal naive is not a weak method that happened to get lucky; by construction of the series, it is very close to the best possible forecast. On real data, where the seasonal shape shifts and shocks occur, it would be less formidable, and a learned model would have more to add.

Discussion question 2. The test months are the future, and their demand level is higher than anything in the training set. Which method suffers from that, and why?

Answer. The network suffers. Its inputs in the test months are lags that reach 470 units and labels that reach 486, while the rows used to fit the weights end in January 2018, when demand had never exceeded about 390 units. The network is being asked to *extrapolate*, to produce sensible outputs for inputs it has never seen, and networks built from ReLU units are not good at that: beyond the training range their response is whatever straight-line continuation the last active units happen to give, which may point in the right direction or not. The seasonal naive has no such problem, because it does not model anything: it carries last year's level forward automatically, so a rising level costs it only one year of drift. The naive forecast likewise carries the level forward, but from only one month ago, so it misses the seasonal swings instead.

Discussion question 3. We trained on 182 rows (145 after the validation split). Is that a lot of data for a network with 5,825 weights?

Answer. No, it is very little. 182 rows for 5,825 weights is about 32 weights per example; 145 rows makes it about 40. Compare the regression examples of Part 5, where fitting two parameters to five or six data points already felt thin. Here the ratio is far worse, in the other direction: many more unknowns than observations. The network can nevertheless learn something because the examples are highly structured and the weights are not all independent, but the shortage of data is exactly why the validation loss sits above the training loss, why early stopping is needed, and why the scorecard numbers jump around from one training run to the next. A larger network would not help; more data, or a smarter question, would.

Lecture link. This is the lesson of Lecture 1, Part 2, in practice. A model is only as good as the baseline it beats, and the baseline must be the strongest free one you can think of. A network that beats naive but not seasonal naive has not yet justified its existence on this series.

Challenge — Stand on the Baseline's Shoulders (Cell 18, YOUR TURN)

Practitioners rarely ask a network to forecast demand from scratch. They ask it to forecast *what the baseline misses*. The forecast becomes

$$\hat{A}(t) = A(t-12) + f(x(t); W),$$

that is, the seasonal naive forecast plus a correction that the network learns. Nothing about the network changes: same 24 inputs, same layers, same optimizer, same loss. Only the *target* changes. Instead of "predict this month's demand", the network is trained to "predict how far last year's value will be off". That difference, demand minus the seasonal naive forecast, is called the *residual*.

Cell 18 — YOUR TURN (one blank)

```
# Train the SAME network on the residual target: what seasonal naive gets wrong
res_train = train_labels - train_dataset['    '].astype('float32')

model_res = keras.Sequential([
    normalizer,
    layers.Dense(64, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1),
])
model_res.compile(optimizer=keras.optimizers.Adam(0.01), loss='mean_absolute_error')
model_res.fit(train_features, res_train, epochs=300, validation_split=0.2,
              callbacks=[keras.callbacks.EarlyStopping(monitor='val_loss', patience=40,
                                                       restore_best_weights=True)],
              verbose=0)

pred_res = snaive + model_res.predict(test_features, verbose=0).flatten()

print(f"Seasonal naive          : MAD = {mad(actual, snaive):6.2f}    MAPE = {mape(actual,
snaive):5.2f}%")
print(f"Network (from scratch): MAD = {mad(actual, pred):6.2f}    MAPE = {mape(actual,
pred):5.2f}%")
print(f"Baseline + network      : MAD = {mad(actual, pred_res):6.2f}    MAPE = {mape(actual,
pred_res):5.2f}%")
```

Line by line: **res_train** is the residual for every training row, the label minus the seasonal naive forecast for that row, which is one of the lag columns (the blank). **model_res** is a second network with exactly the architecture of Cell 10, compiled the same way. It is trained on the same inputs but with **res_train** as the answers, with the same early-stopping rule. Its forecast for the test months, **pred_res**, is the seasonal naive forecast plus the network's predicted correction. The three print lines put the seasonal naive, the from-scratch network of Cell 13, and the combination side by side. This cell trains a network, so it takes twenty to forty seconds.

Blank: The residual is demand minus the seasonal naive forecast. Which lag column is the seasonal naive forecast? You already used it in Cell 13.

Expected output. Three lines. The seasonal naive and the from-scratch network repeat their Cell 13 scores. The combination typically lands at a MAD of about 8 to 9 units (anywhere from about 7.5 to 9.5 is normal), the first model in the notebook to beat the seasonal naive, and it does so reliably from one run to the next.

Checkpoint. The CHECK cell requires the average size of the residual to be small (below 20 units). If you subtracted the wrong column, the residual is large and the message says so. It prints "✓ Cell 18 check passed".

Question. Why does "baseline plus network" beat both the network alone and the seasonal naive alone?

Answer. Because of a division of labor. The baseline supplies the level and the seasonal shape for free, and it supplies them at the *right level*, since last year's value already sits near this year's, so the extrapolation problem of Question 2 disappears. What is left for the network is a small target: the year-over-year drift of about 10 units plus whatever pattern remains in the residual. Small targets produce small errors. In the from-scratch model, a 3 percent mistake on a demand of 450 units is 13 units; in the residual model, even a 30 percent mistake on a residual of 10 units is 3 units. The network's inputs are the same, but its mistakes are now mistakes on a much smaller number. And because the residual does not trend upward the way demand does, the test-month residuals lie inside the range the network trained on, so the network is interpolating rather than extrapolating.

Question. The associative-forecasting slide says that associative models are also used to correct time-series models. How is that sentence realized here?

Answer. Literally. The seasonal naive forecast is a time-series method: it uses only the series' own past. The network is an associative model: it relates inputs (lags and month switches) to an output. In Cell 18 the associative model is not replacing the time-series method; it is correcting it, adding an adjustment on top of a forecast that already carries level and season. This is how neural networks are most often used in practice for demand forecasting: as a correction to a strong, simple baseline, not as a replacement for one.

Question. Did we change the network in any way to get this improvement?

Answer. No. The architecture, the inputs, the normalizer, the optimizer, the learning rate, the loss, and the early-stopping rule are all identical to Cell 10 and Cell 11. The only thing that changed is the column of answers the network was trained on. This is worth remembering: the biggest improvement of the day came not from a bigger or cleverer model but from asking the same model a smarter question.

Cells 19 and 20 — The Experiment Helper (optional)

Cell 19 — Experiment helper (provided; nothing to fill in)

Cell 19 defines a function, `train_and_score`, that repeats the whole notebook in one call: it rebuilds the training table from `raw`, splits it by time, normalizes, builds the lecture network, trains it with the chosen settings, and prints a scorecard and a learning curve. It exists so that an experiment is a single line. The cell is provided in full and has no blanks; run it once exactly as it is. Its first line and its last line are the only ones you need to read closely:

Cell 19 (first and last lines)

```
def train_and_score(window=12, use_month=True, lr=0.01, early_stopping=True, seed=510,
    show=True):
    ... # rebuilds the table, trains the lecture network, prints the scorecard and the
    learning curve

baseline_run = train_and_score() # the lab configuration, for reference
```

The five settings, with their default values in parentheses, are:

- `window` (12): how many lag months the network sees.
- `use_month` (True): whether the twelve month switches are included among the inputs.
- `lr` (0.01): the learning rate of Adam.
- `early_stopping` (True): whether the early-stopping rule is used. If it is switched off, the helper trains for a fixed 200 epochs and keeps the final weights.
- `seed` (510): the random seed, so that a run can be repeated.

Calling `train_and_score()` with no arguments reproduces the lab configuration and stores the result in `baseline_run`. It prints one header line, for example `window=12 use_month=True lr=0.01 early_stopping=True | epochs run: 78`, then three scorecard lines (network, naive, seasonal naive), and then the learning curve. The network's score will be close to, but not identical with, your Cell 13 result, because it is a fresh training run.

Cell 20 — Optional: try one change

Cell 20

```
# Example – replace the argument with the one you chose:
my_run = train_and_score(lr=0.001)
```

If you are ahead of pace, or curious after class, change **exactly one** setting, run the cell, and compare its printed block and learning curve with the reference run from Cell 19. For example, `train_and_score(window=3)` trains the same network with a three-month window. Ask yourself which lecture concept (the learning rate, the seasonal inputs, the window length, or overfitting and early stopping) explains what you see. A single training run is a noisy measurement, so if a result surprises you, run it again with `seed=1` added (for example `train_and_score(window=3, seed=1)`) and check whether it holds.

Tip. Keep this notebook. Choose File › Save a copy in Drive now if you have not already. A follow-up assignment will build on it (details will be posted on Canvas), and the final group project extends this very file: classical methods (exponential smoothing, DES, TES, regression) against this network, on the same data, with the same MAD and MAPE scorecard on the same held-out months.

Part E — Reading Your Results Like a Forecaster

E1. What a typical screen says

After Cells 13 and 18 your scorecard should look roughly like the table below. The two baselines are exact; the two networks vary from run to run.

Forecast	MAD (units)	MAPE	Comment
Naive (copy last month)	30.6	8.4%	Exact. Misses every seasonal turn by one month.
Network, from scratch (Cell 13)	about 12 to 16	about 3 to 4.5%	Varies by run. Half the naive error; slightly worse than the seasonal naive.
Seasonal naive (copy the same month last year)	11.2	2.9%	Exact. Right shape, slightly low because of one year of trend.
Baseline plus network (Cell 18)	about 8 to 9	about 2 to 2.5%	Varies by run. The first model to beat the seasonal naive, and it does so reliably.

E2. How to state a result

Part 2 of the lecture taught a way of reporting forecast accuracy; use it whenever you report a result. A good sentence names the hold-out period, the measure, the number, and the comparison, all in one breath. For example: "On the 46 held-out months (March 2021 to December 2024), the network's MAPE is about 3.4 percent, within a few units of the seasonal naive forecast and less than half the naive error." Or, for the challenge: "Trained on the residual, the network reduces the seasonal naive's MAD from 11.2 to about 8 units on the same 46 months, an improvement of roughly a quarter." Notice what these sentences avoid: a bare number with no baseline, a number with no period, and decimals that pretend to a precision that a random training run cannot support.

E3. Why rank order matters more than decimals

Every trained network is one random draw from the many networks that gradient descent could have produced from the same data. Your neighbor's draw differs from yours. So the statements that survive from run to run are statements about *rank*: the network beats naive by about a factor of two; the seasonal naive edges out the from-scratch network; the residual model beats everything. If you need a number you can defend, run the training more than once (the helper in Cell 19 takes a **seed** argument for exactly this purpose) and report the range you saw, not a single draw.

Part F — Troubleshooting

What you see	What it means	What to do
<code>NameError: name '____' is not defined</code>	A blank in this cell is still four underscores.	Fill every blank in the cell, then run it again.
<code>KeyError: '____'</code>	A blank that should hold a column name is still empty.	Fill it with the column name, inside quotation marks.
<code>KeyError: 'lag_13'</code> or another column name	The code asked for a column that does not exist in the table.	Check the spelling and the number in the name; column names are <code>lag_1</code> to <code>lag_12</code> and <code>m_1</code> to <code>m_12</code> .
<code>NameError: name 'dataset' is not defined</code> (or <code>model</code> , <code>train_features</code> , <code>mad</code> , and so on)	The cell that creates that name has not been run in this runtime, for example after a restart.	Run the earlier cells from the top, then this one. Runtime › Run all does this, stopping at the first empty blank.
<code>FileNotFoundError: ... 'demand_monthly.csv'</code>	The data file is not in the runtime, or has a different name.	Run Cell 3 again and upload the file; rename it if your browser added a suffix such as "(1)".
<code>SyntaxError: invalid syntax or unterminated string literal</code>	A quotation mark or parenthesis went missing while you filled a blank.	Compare the line with the code box in this guide; text answers must be inside quotation marks, numbers must not.
<code>AssertionError: expected 12 lag columns – check range(1, N)</code>	Blank 1 of Cell 5 is off by one.	Remember that <code>range(1, N)</code> stops before N.
<code>AssertionError: the test set must come AFTER the training set</code>	The two slices in Cell 6 are swapped.	The past goes to <code>train_dataset</code> , the future to <code>test_dataset</code> .
<code>AssertionError: expected 5,825 trainable weights ...</code>	The output layer has the wrong number of units, or a hidden layer size was changed.	The output layer predicts one quantity; the hidden layers stay at 64.
<code>AssertionError: naive should copy LAST month or ... the same month LAST YEAR</code>	A baseline in Cell 13 points at the wrong lag column.	Last month is one month back; the same month last year is twelve months back.
<code>AssertionError: the residual should be small</code>	Cell 18 subtracts the wrong column.	Subtract the seasonal naive forecast, the same column you used for <code>snaive</code> in Cell 13.
Cell 11 or Cell 18 shows a spinning icon for a long time	Training is running; it normally takes 20 to 40 seconds, longer on a busy day.	Wait. If it exceeds a few minutes, choose Runtime › Interrupt execution and run the cell again.
A message that the runtime has disconnected or was restarted	The cloud computer went away (inactivity, or a long session). Everything in memory is gone.	Runtime › Run all; upload the CSV again when Cell 3 asks for it.
<code>ModuleNotFoundError: No module named 'google.colab'</code>	You are running the notebook outside Colab (for example in a local Jupyter).	Put <code>demand_monthly.csv</code> in the same folder as the notebook; the upload step is then skipped automatically.

What you see	What it means	What to do
Cell 16 prints "Export skipped"	The optional conversion library is not available.	Nothing to fix; the cell is optional.
Plots do not appear	The cell ran but the figure was not displayed, usually because <code>plt.show()</code> was removed.	Run the cell again unchanged.
Your numbers differ from this guide or from your neighbor	Training is random; see Part A5.	Nothing to fix. Compare the ranking, not the decimals.

Part G — Vocabulary and What Is Examinable

G1. The ten code words of the day, mapped to the lecture

None of the words in the left column is on the exam. Every concept in the right column is.

Code word	What it does in the notebook	The lecture concept it stands for
<code>DataFrame</code>	A table in memory with named columns	The spreadsheet of data: the series, then the regression table of Part 5
<code>shift(k)</code>	The demand column moved down by k rows	A lag, $A(t-k)$; twelve of them are the sliding window
<code>get_dummies</code>	Twelve 0/1 columns, one per month	The seasonal switch: a categorical predictor as dummy variables, one-hot
<code>iloc[:split], iloc[split:]</code>	Rows before and from the cut	The split by time: past for training, future for testing
<code>Normalization</code>	Rescale each input to mean 0, s.d. 1	Scaling the inputs, as in the "scaled by 100" hand example
<code>Sequential</code>	A stack of layers	The architecture picture: input, hidden layers, output
<code>Dense(64, activation='relu')</code>	64 units computing $g(w^T x + b)$	The perceptron, repeated; ReLU as g
<code>compile(optimizer, loss)</code>	Choose the objective and the optimizer	$J(W)$ as MAD or MSE; gradient descent with learning rate η
<code>fit(...)</code>	The training loop	Epochs, mini-batches, backpropagation, early stopping on the validation loss
<code>predict(...)</code>	Compute outputs for new rows	The forward pass, the calculation you did by hand

G2. Glossary of ideas

Term	Meaning
Lag	A value of the series from an earlier month. <code>lag_k</code> in the row for month t holds $A(t-k)$.
Sliding window	The twelve most recent months, viewed as one example; moving it forward by one month gives the next example. The twelve lag columns are all positions of the window at once.
One-hot encoding	Representing a category (the month) as a set of switches with exactly one switch on.
Label	The answer that goes with an example; here, the demand of the month the window points at.
Features	The inputs of an example; here, the twelve lags and the twelve switches.
Training, validation, test sets	Three chronological blocks: the rows used to update the weights; the rows used to decide when to stop; the rows used once, at the end, for the score.
Normalization	Rescaling each input to mean 0 and standard deviation 1 using statistics from the training rows.

Term	Meaning
Perceptron, unit	One node of the network: a weighted sum of its inputs plus a bias, passed through an activation function.
ReLU	The activation $g(z) = \max(0, z)$: output 0 for negative z , otherwise z itself.
Loss, objective $J(W)$	The formula that measures how wrong the network is on the training rows; here the mean absolute error. Training minimizes it.
Gradient descent	Repeatedly step the weights in the direction that lowers the loss, by a step of size η (the learning rate).
Adam	A version of gradient descent that adapts the step size for each weight as training proceeds.
Backpropagation	The algorithm that computes the gradient of the loss with respect to every weight, layer by layer from the output backward, using the chain rule.
Epoch	One complete pass over the training rows.
Mini-batch	A group of rows (32 here) processed together for one weight update; an epoch of 145 rows is five updates.
Learning curve	The training loss and the validation loss plotted against the epoch.
Overfitting	Fitting the noise of the training rows: the training loss keeps falling while the validation loss stops improving or rises.
Early stopping	Stop training when the validation loss has not improved for a set number of epochs, and keep the weights from the best epoch.
Forward pass	Computing the network's output for a given input, layer by layer.
Baseline	A forecast that needs no model: naive (copy last month) and seasonal naive (copy the same month last year).
MAD, MAPE	Mean absolute deviation, in units; mean absolute percentage error, in percent. Both from Lecture 1, Part 2.
Residual	What the baseline gets wrong: demand minus the seasonal naive forecast. The challenge trains the network on it.
Extrapolation	Producing outputs for inputs outside the range seen in training; hard for networks, automatic for the seasonal naive.

G3. What is examinable

The exams assess models, algorithms, and pseudocode, not software syntax. From this lab, the examinable ideas are:

- the forward pass, computed by hand for a small network;
- the loss (mean absolute or mean squared error), gradient descent, and the learning-rate trade-off (too small converges slowly, too large overshoots);
- what backpropagation computes (the gradient of the loss with respect to every weight);
- overfitting, and early stopping as a remedy;
- windowing (lags) and one-hot inputs, and why a time-series split must be by time, with the most recent months as the test set;

- why a forecast is judged against a baseline, and how to state a result in terms of MAD and MAPE on a held-out period.

Not examinable: any Python or Keras syntax. Next class we move to linear programming, where this forecast becomes the demand row of an optimization model.